5️⃣ 로키스터디 5주차

> 네트워크 기본 개념과 인터페이스 구성 이해, nmcli·nmtui를 활용한 IP/게이트웨이/DNS 설정, 네트워크 상태 점검 도구 활용(ping, ss, ip), 방화벽(firewalld) 정책 설정, SELinux 동작 방식과 기본 보안 정책 이해

- 네트워크 기본 개념과 인터페이스 구성 이해
  - 운영체제와 네트워크의 구분
  - 네트워크 인터페이스
    - 대표 인터페이스
  - 인터페이스 상태
  - 인터페이스에 필요한 핵심 설정
  - NetworkManager란?
  - 연결 프로필
  - 연결 프로필 저장 방식
  - 네트워크를 붙이는 방식
- nmcli·nmtui를 활용한 IP/게이트웨이/DNS 설정
  - 인터페이스와 연결 프로필
  - IP
  - 게이트웨이
  - DNS
  - nmcli
  - nmtui
  - 실습 1 - nmcli를 활용한 고정 IP / 게이트웨이 / DNS 설정 실습 정리
  - 실습 2 - nmtui를 활용한 고정 IP / 게이트웨이 / DNS 설정 실습
- 네트워크 상태 점검 도구 활용
  - 1. `ip` : 가장 기본적인 상태 점검 도구
    - 1-1. `ip address`
    - 1-2. `ip route`
    - 1-3. 정책 라우팅까지 갈 때의 `ip`
  - 2. `ping` : 실제로 닿는지 확인하는 도구
    - 2-1. 게이트웨이로 `ping`
    - 2-2. 외부 IP로 `ping`
    - 2-3. 도메인으로 `ping`
  - 3. `ss` : 어떤 서비스가 어떤 포트에서 듣고 있는지 확인
  - 4. DNS 상태 점검: `/etc/resolv.conf`, `ss`, `journalctl`, `tcpdump`
    - DNS 흐름
    - 4-1. `/etc/resolv.conf`
    - 4-2. `journalctl`
    - 4-3. `tcpdump`
    - 실습
- 방화벽 정책 설정
  - 방화벽 정책 설정이 필요한 이유
  - zone
  - service
  - port
  - rich rule
  - runtime
  - permanent
  - policy object
  - 서비스 허용과 포트 허용의 차이
  - 실습
- SELinux 동작 방식과 기본 보안 정책 이해
  - SELinux란
  - 동작 방식
  - 1. Label
  - 2. Domain과 Type
  - 기본 보안 정책 - Targeted 정책
  - 정책 원칙
  - SELinux 모드
  - 자주 발생하는 문제 상황
  - Boolean
  - 문제 해결 흐름
    - 주요 명령어
  - 실습
    - 1. SELinux 상태 확인
    - 2. permissive 모드로 전환 테스트
    - 3. 다시 enforcing 모드 복귀
    - 4. 파일 label 확인
    - 5. label 복구
    - 6. Boolean 설정 확인 및 변경
    - 7. 포트 허용 확인
    - 요약

리눅스 서버를 네트워크에 연결하고, 통신 상태를 확인하고, 외부 접근을 통제하고, OS 차원에서 추가 보안을 거는 흐름

서버가 네트워크를 쓰려면 우선 **네트워크 인터페이스**가 있어야 하고, 그 인터페이스에 **IP 주소**, **게이트웨이**, **DNS** 같은 설정이 붙어야 한다. 그다음 실제로 통신이 되는지 `ip`, `ping`, `ss` 같은 도구로 확인해야 한다. 그런데 통신이 된다고 해서 아무 접근이나 다 허용하면 안 되니까 **firewalld**로 들어오고 나가는 트래픽을 제어한다. 그래도 여전히 프로세스가 파일이나 포트, 서비스 자원에 과하게 접근할 수 있으므로, 마지막으로 **SELinux**가 OS 내부에서 "이 프로세스가 이 행동을 해도 되는가"를 한 번 더 검사한다. 즉, 네트워크 설정은 연결의 문제이고, 방화벽은 트래픽 허용의 문제이고, SELinux는 프로세스 권한 통제의 문제다.

네트워크 기본 개념과 인터페이스 구성 이해

학교에서 배웠던 네트워크 기본 개념

```
[ Application Layer ]  →  데이터 생성
        ↓
[ Transport Layer ]   →  세그먼트화, 신뢰성 보장 (TCP/UDP)
        ↓
[ Network Layer ]     →  IP 주소 기반 라우팅
        ↓
[ Data Link Layer ]   →  프레임화, MAC 주소로 구분
        ↓
[ Physical Layer ]    →   전기/광 신호로 비트 전송
```

```
내가 5계층의 어플리케이션 (카카오톡)을 사용하며 엄마에게 문자를 보내고싶어한다고 가정하면
(L2) 우리의 목표는 MAC주소를 통해 라우터 이동하면서
(L3) 라우팅 하다가 엄마 폰의 IP를 찾으면 나의 destination은 찾은 거라고 볼 수 있다
(L4) 근데 내가 엄마한테 메시지를 보낼때 데이터가 inorder로
    error없이 올바르게 도착하는걸 보장할건지 말건지에 따른 통신 방법을 고를 수 있어서
    정해진 프로토콜에 따라 stateless할건지 stateful할건지 알 수 있다.
    또한 포트번호를 통해 어떤 application인가까지 파악 가능하다 "TCP 3번 포트"
    (여기서 소켓이 열린다 )
(L5) 이제 어느 단말기인지 3계층이 찾아줬고 포트번호를 통해 application 어디에 있는지 알고
    어떤 통신 방법을 쓸건지도 확인했으니까 (준비 끝, 4계층이 소켓을 열면서 소통을 위한 문이 열린 것)
    카카오톡과 같은 응용 프로그램은 이 계층에서 실행되어 application message (msg)를 생성
```

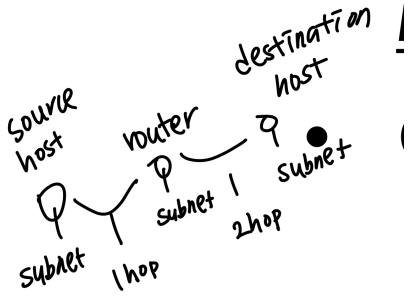
1. Physical Layer

데이터 전송 장치(예: 컴퓨터)와 **전송 매체 또는 네트워크** 사이의 **물리적인 연결(interface)** 을 다루는 계층

- 전송 매체의 특성
- 시그널 본연의 성질
- Data Rate

2. Data Link Layer

- 이더넷이나 Wifi 같은 네트워크 기술을 통해 실제로 데이터를 다음 노드 (다음 hop)까지 전달해주는 역할
- 다음 hop 이동까지 address 주소 제공 : MAC
- 중요도에 따라 우선순위 정해서 처리한다.
- Shared medium이었다면 switch가 아닌 hub였다면 딱 1개의 노드만 받아야할 때 이때 필요한게 MAC 인 것
- 같은 네트워크 사이에 있는 장치들 간의 통신 담당 (한 홉안의 모든 L2는 같은 프로토콜인 것) = 같은 subnet part를 가짐. IP 앞에가 같음
- 한 홉을 delivery 한다면 subnet을 지나가는 것
- 라우터 입장에서는 한 subnet에서 다른 subnet으로 이동하는 것이다.
- router는 여러 개의 2계층 프로토콜을 가짐
- 어떤 네트워크를 쓰느냐에 따라 이 계층에서 쓰는 프로토콜이 달라진다.
- 상위 계층은 내가 어떤 네트워크를 쓰는지 몰라도된다. (유선인지 무선인지 관심이 없음)



3. Network Layer (host to host)

- 다수 상호 연결된 네트워크를 통해 데이터가 이동할 수 있도록 동작을 수행한다.
- routing 개념을 가진다.
- end system, router 가짐 router만 routing한다고 봄
- host는 라우팅 경로 계산 안한다. 무조건 가까이 잇는 디폴트 라우터에게 넘겨서 목적지를 찾아가는 경로를 계산한다.
- 라우터는 서로 다른 네트워크를 연결해준다고 볼 수 있다. 그래서 다양한 2계층 프로토콜을 가짐!
- 보통 NIC에 IP 주소가 할당된다. 각 기기 NIC에 고유한 IP 주소가 할당되는 것이지 운영체제에 IP 주소가 할당되는 것은 아니다.
- 둘 이상의 NIC 카드를 가지는 경우 서버 호스트 또는 라우터는 둘 이상의 IP 주소를 가질 수 있다.

4. Transport Layer (Process to Process)

- L4는 모든 애플리케이션에서 공통으로 사용되는 계층이다.
- TCP UDP 모두 상위 계층 프로세스를 찾아주는 기능이 존재(port number 이용해서) + Checksum을 이용해서 error detecion 하는 알고리즘 동일
- TCP
  - connection orientes protocol → 양쪽 끝의 TCP가 software적으로 필요한 파라미터들을 세팅한다 = logical association (Data transfer 이전에)
  - reliable (no loss, in order) service 제공
  - loss 에 민감한 응용들이 선호함 : SMTP, FTP, SSH
  - flow control, congestion control
- UDP
  - connection less protocol
  - 부가 기능이 없어 상위 응용에서 보낸 PDU를 곧장 IP에게 내려보내 ㅁ
  - loss보다 delay에 민감한 응용들, 스스로 필요한 기능을 구현하는 smart applicaiton이 선호함

5. Application Layer

- 여러 사용자 응용 프로그램을 지원하는 논리 포함된다
- 각 응용 프로그램 마다 다른 모듈 필요
- 전통적인 응용 VS 멀티미디어응용 구분
  - 기존에는 인터넷이 정보 검색, 이메일, 파일 전송, 웹(텍스트 및 이미지 기반)에 주로 사용되었다.
  - 현재는 오디오 및 비디오 스트리밍처럼 대용량 데이터를 다루는 멀티미디어 응용의 수요가 증가하고 있다.

이번 루키 스터디에서는 리눅스가 네트워크에 붙을 때 **무엇을 어떻게 관리하는지** 이해하는 것이 목표다.

즉, 운영체제가 네트워크 장치를 인식하고, 그 장치에 설정을 적용해 실제 통신 가능한 상태로 만드는 과정을 배우는 단원이다. RHEL 10은 네트워크 장치 이름, 이더넷 연결, NetworkManager 연결 프로필을 중심으로 이를 설명한다.

운영체제와 네트워크의 구분

애플리케이션은 직접 네트워크와 통신하지 않고 운영체제를 통해 통신한다.

- OS가 하는 일
  1. 소켓 생성
  2. TCP/UDP 선택
  3. IP 주소와 포트 기반 통신 처리
  4. 라우팅 결정
  5. 패킷 생성
  6. 네트워크 인터페이스로 전달
- 네트워크가 하는 일
  - 라우터
  - 인터넷
  - 상대방 장치

즉, **운영체제가 패킷을 만들고, 네트워크는 그 패킷을 전달한다.**

네트워크 인터페이스

네트워크 인터페이스는 서버가 네트워크와 연결되는 통로다.

RHEL 10에서는 `udev` 기반의 **일관된 인터페이스 이름 체계**를 사용하며, 기본 naming scheme은 `rhel-10.0`이다. 그래서 예전의 `eth0` 대신 `eno1`, `enp1s0` 같은 이름을 사용한다.

대표 인터페이스

인터페이스	설명	활용
`ens3`, `eno1`, `enp1s0`	실제 물리 NIC 또는 VM의 가상 NIC	외부 네트워크와 통신하는 주 인터페이스
`lo`	루프백 인터페이스	자기 자신과 통신, 로컬 테스트

지금 단계에서는 **호스트 기본 인터페이스와 loopback만 우선 이해하면 충분하다.**

컨테이너용 `docker0`, `br-xxxx`, `veth`는 나중에 Docker/컨테이너 네트워크를 공부할 때 보면 된다.

인터페이스 상태

인터페이스가 있다고 해서 바로 통신이 가능한 것은 아니다.

장치가 연결되어 있는지, NetworkManager가 관리하는지 확인해야 한다. `nmcli device status`에서는 `connected`, `disconnected`, `unmanaged` 같은 상태를 볼 수 있다.

인터페이스에 필요한 핵심 설정

네트워크 인터페이스를 실제로 사용하려면 다음 설정이 필요하다.

- **IP 주소**: 이 장치의 네트워크 주소
- **게이트웨이**: 다른 네트워크로 나갈 때 거치는 출구
- **DNS**: 이름을 IP 주소로 바꿔주는 설정

즉, 리눅스는 단순히 NIC를 인식하는 것이 아니라,

이 인터페이스에 어떤 주소를 쓰고, 어디로 나가고, 이름을 어떻게 해석할지를 관리한다.

NetworkManager란?

NetworkManager는 RHEL에서 네트워크 연결을 관리하는 서비스다.

이더넷, IP 주소, 라우팅, DNS 같은 설정을 **연결 프로필** 단위로 관리하며, `systemctl`로 서비스 상태를 볼 수 있고 `nmcli`로 설정을 제어할 수 있다. `nmcli`는 연결을 생성·수정·활성화·비활성화하고 장치 상태를 표시하는 도구다.

연결 프로필

RHEL 10에서는 네트워크 설정을 **연결 프로필(connection profile)** 로 관리한다.

NetworkManager는 호스트의 각 이더넷 어댑터에 대해 연결 프로필을 만들고, 기본적으로 IPv4/IPv6에 DHCP를 사용한다. 필요하면 정적 IP 설정용 프로필을 새로 만들거나 기존 프로필을 수정할 수 있다.

즉,

- **인터페이스** = 실제 네트워크 장치
- **연결 프로필** = 그 장치에 적용할 설정 묶음

이라고 이해하면 된다.

연결 프로필 저장 방식

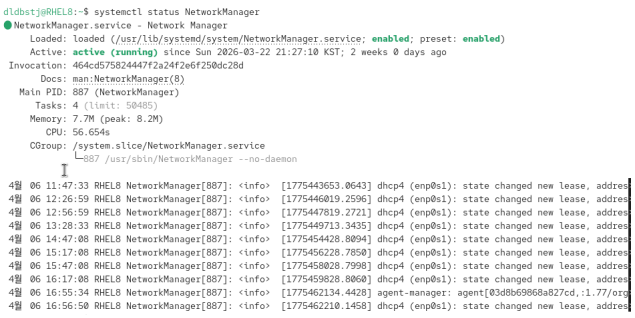
RHEL 10에서는 연결 프로필이 **keyfile 형식**으로 저장되며, 영구 프로필은 `/etc/NetworkManager/system-connections/` 아래 `.nmconnection` 파일로 관리된다. Red Hat은 이 파일을 직접 수정하기보다 `nmcli`, RHEL system role, `nmstate` 같은 도구로 관리할 것을 권장한다.

네트워크를 붙이는 방식

1. **DHCP**
   - DHCP 서버가 IP 주소, 게이트웨이, DNS를 자동으로 내려주는 방식
2. **정적 IP**
   - 관리자가 직접 IP 주소, 프리픽스, 게이트웨이, DNS를 지정하는 방식

RHEL에서는 이 둘 모두 연결 프로필 설정 방식의 차이로 관리된다

```
systemctl status NetworkManager
```



nmcli·nmtui를 활용한 IP/게이트웨이/DNS 설정

위에서 언급했던 인터페이스를 연결하기 위해서 설정을 관리하는 연결프로필이라는걸 만드는 NetworkManager가 이 명령어들을 통해 조작한다

인터페이스와 연결 프로필

- 인터페이스(device)는 실제 NIC이고 enp1s0같은 이름으로 보임
- 연결 프로필(connection)은 그 NIC에 적용할 설정 묶음

device와 connection은 분리된 관계

같은 NIC이라고 해도 학교용, 집 네트워크용, 고정 IP 실습용으로 프로필을 여러개 둘 수 있음

“인터페이스가 켜져 있다” = OS가 그 NIC을 사용 가능한 상태로 만들어놨다

IP

- NIC에 부여되는 논리 주소
- `192.168.0.10/24`에서 `/24`는 서브넷 범위

게이트웨이

같은 네트워크 안에서는 게이트웨이 없이 통신이 가능하지만 인터넷이나 다른 대역으로 나가기 위해선 기본 게이트 웨이가 필요함

- 내 서브넷 밖으로 나갈 때 다음으로 넘기는 라우터

DNS

- `google.com` 같은 이름을 IP로 바꿔주는 서버

nmcli

- 명령줄 기반 CLI

nmtui

- 텍스트 기반 UI
- 방향키, 엔터, space를 통해 연결하는 것

실습 1 - nmcli를 활용한 고정 IP / 게이트웨이 / DNS 설정 실습 정리

- 실습 전체 흐름

```
현재 상태 확인
→ connection 확인
→ static IP 설정 (manual)
→ connection 재적용
→ 결과 검증 (ip / route / DNS / ping)
→ DHCP로 복구 (optional)
```

- 현재 네트워크 상태 확인

```
dldbstj@RHEL8:~$ nmcli device status
nmcli connection show
ip addr
ip route
cat /etc/resolv.conf
DEVICE  TYPE      STATE          CONNECTION
enp0s1  ethernet  연결됨         enp0s1
lo      loopback  연결됨 (외부)   lo
NAME    UUID                                  TYPE      DEVICE
enp0s1  80845e61-5a51-3cba-a0e1-3ed8c641e6ad  ethernet  enp0s1
lo      30250a86-1d6c-4b51-9da1-983b34417cf4  loopback  lo
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 22:5a:f8:47:24:24 brd ff:ff:ff:ff:ff:ff
    altname enx225af8472424
    inet 192.168.64.3/24 brd 192.168.64.255 scope global dynamic noprefixroute enp0s1
       valid_lft 3049sec preferred_lft 3049sec
    inet6 fd7e:7ced:7640:9bde:205a:f8ff:fe47:2424/64 scope global dynamic mngtmpaddr proto kernel_ra
       valid_lft 2591952sec preferred_lft 604752sec
    inet6 fe80::205a:f8ff:fe47:2424/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
default via 192.168.64.1 dev enp0s1 proto dhcp src 192.168.64.3 metric 100
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.3 metric 100
# Generated by NetworkManager
nameserver 192.168.64.1
```

```
nmcli device status
nmcli connection show
ip addr
ip route
cat /etc/resolv.conf
```

- 고정 IP 설정

```
sudo nmcli connection modify"Wired connection 1" \
  ipv4.method manual \
  ipv4.addresses192.168.0.50/24 \
  ipv4.gateway192.168.0.1 \
  ipv4.dns"8.8.8.8 1.1.1.1"
```

  - 각 옵션 의미

항목	의미
ipv4.method manual	DHCP → 수동 설정
ipv4.addresses	내 IP + 서브넷
ipv4.gateway	기본 게이트웨이
ipv4.dns	DNS 서버

  - CIDR 반드시 포함하고 gateway는 같은 네트워크가 되도록 !

- 설정 적용

```
sudo nmcli connection up"Wired connection 1"
```

- 적용 결과 확인

```
ip addr show
ip route
cat /etc/resolv.conf
```

```
dldbstj@RHEL8:~$ sudo nmcli connection modify "enp0s1" \
  ipv4.method manual \
  ipv4.addresses 192.168.64.50/24 \
  ipv4.gateway 192.168.64.1 \
  ipv4.dns "8.8.8.8 1.1.1.1"

sudo nmcli connection up "enp0s1"
[sudo] dldbstj 암호:
연결이 성공적으로 활성화되었습니다 (D-버스 활성 경로: /org/freedesktop/NetworkManager/ActiveConnection/3)
dldbstj@RHEL8:~$ ip addr
ip route
cat /etc/resolv.conf
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 22:5a:f8:47:24:24 brd ff:ff:ff:ff:ff:ff
    altname enx225af8472424
    inet 192.168.64.50/24 brd 192.168.64.255 scope global noprefixroute enp0s1
       valid_lft forever preferred_lft forever
    inet6 fd7e:7ced:7640:9bde:205a:f8ff:fe47:2424/64 scope global dynamic mngtmpaddr proto kernel_ra
       valid_lft 2591990sec preferred_lft 604790sec
    inet6 fe80::205a:f8ff:fe47:2424/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
default via 192.168.64.1 dev enp0s1 proto static metric 100
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.50 metric 100
# Generated by NetworkManager
nameserver 8.8.8.8
nameserver 1.1.1.1
```

- Before: DHCP 기반 자동 IP (192.168.64.3)
  After: 수동(static) 설정 IP (192.168.64.50)
- IP가 정상적으로 192.168.64.50으로 변경됨 → 고정 IP 설정 성공
- default route 존재 확인
  → default via 192.168.64.1 dev enp0s1 proto static
- DNS 설정 반영 완료
  → 이전: 192.168.64.1 (공유기 DNS)
  → 현재: 8.8.8.8, 1.1.1.1 (수동 설정 DNS)

- 네트워크 테스트

```
dldbstj@RHEL8:~$ ping -c 3 192.168.64.1
PING 192.168.64.1 (192.168.64.1) 자료의 56(84) 바이트.
64 바이트 (192.168.64.1에서): icmp_seq=1 ttl=64 시간=3.98 ms
64 바이트 (192.168.64.1에서): icmp_seq=2 ttl=64 시간=0.392 ms
64 바이트 (192.168.64.1에서): icmp_seq=3 ttl=64 시간=0.579 ms

--- 192.168.64.1 ping 통계 ---
3 패킷이 전송됨, 3 수신됨, 0% packet loss, time 2065ms
rtt min/avg/max/mdev = 0.392/1.651/3.984/1.650 ms
dldbstj@RHEL8:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 자료의 56(84) 바이트.
64 바이트 (8.8.8.8에서): icmp_seq=1 ttl=114 시간=37.5 ms
64 바이트 (8.8.8.8에서): icmp_seq=2 ttl=114 시간=35.2 ms
64 바이트 (8.8.8.8에서): icmp_seq=3 ttl=114 시간=52.4 ms

--- 8.8.8.8 ping 통계 ---
3 패킷이 전송됨, 3 수신됨, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 35.192/41.690/52.420/7.643 ms
dldbstj@RHEL8:~$ ping -c 3 google.com
PING google.com (172.217.209.101) 자료의 56(84) 바이트.
64 바이트 (hq-in-f101.1e100.net (172.217.209.101)에서): icmp_seq=1 ttl=110 시간=34.0 ms
64 바이트 (hq-in-f101.1e100.net (172.217.209.101)에서): icmp_seq=2 ttl=110 시간=41.3 ms
64 바이트 (hq-in-f101.1e100.net (172.217.209.101)에서): icmp_seq=3 ttl=110 시간=42.8 ms

--- google.com ping 통계 ---
3 패킷이 전송됨, 3 수신됨, 0% packet loss, time 2007ms
rtt min/avg/max/mdev = 34.003/39.348/42.768/3.828 ms
```

```
ping-c3192.168.0.1# 게이트웨이
ping-c38.8.8.8# 외부 IP
ping-c3 google.com# DNS 확인
```

- gateway(192.168.64.1) ping 결과 0% packet loss로 정상 응답을 확인하였다.
  → 내부 네트워크 통신이 정상적으로 동작함
- 외부 IP(8.8.8.8) ping 결과 정상 응답을 확인하였다.
  → 인터넷 연결이 정상적으로 이루어짐
- 도메인(google.com) ping 결과 정상 응답을 확인하였다.
  → DNS를 통한 이름 해석이 정상적으로 동작함
- 의미

테스트	확인 내용
gateway ping	내부 네트워크 연결
8.8.8.8 ping	인터넷 연결
google ping	DNS 정상 여부

- DHCP로 복구

```
sudo nmcli connection modify"Wired connection 1" ipv4.method auto
sudo nmcli connection up"Wired connection 1"
```

실습 2 - nmtui를 활용한 고정 IP / 게이트웨이 / DNS 설정 실습

- 실습 전체 흐름

```
nmtui 실행
→ connection 선택/수정
→ IPv4 설정 manual 변경
→ IP / gateway / DNS 입력
→ 저장 후 적용
→ 결과 검증
```

- nmtui 실행

```
sudo nmtui
```

- 연결 수정 메뉴 진입
  - `Edit a connection` 선택
  - 현재 사용 중인 connection 선택 (예: `Wired connection 1`)
  - Enter
- IPv4 설정 변경
  - `IPv4 CONFIGURATION` 항목 이동
  - `Automatic` → `Manual`로 변경
  - `Show` 선택해서 상세 설정 열기
- 고정 IP / 게이트웨이 / DNS 입력
  - Addresses: `192.168.0.50/24`
  - Gateway: `192.168.0.1`
  - DNS servers: `8.8.8.8, 1.1.1.1`
  - Search domains: (옵션) `lab.local`
- 설정 저장 : `OK` → `Back` → `Quit`
- 설정 적용

```
nmcli connection up"Wired connection 1"
```

- 적용 결과 확인

```
ip addr
ip route
cat /etc/resolv.conf
```

- 네트워크 테스트

```
ping-c3192.168.0.1
ping-c38.8.8.8
ping-c3 google.com
```

- DHCP로 복구

```
sudo nmtui
```

  - 다시 `Edit a connection`
  - IPv4 CONFIGURATION → `Automatic` 변경
  - 저장 후 적용

네트워크 상태 점검 도구 활용

1. `ip` : 가장 기본적인 상태 점검 도구

RHEL 문서에서 가장 자주 확인하는 도구

- `ip address show <인터페이스>`
- `ip route show default`
- `ip -6 route show default`

1-1. `ip address`

인터페이스 상태와 할당된 IP 주소를 본다.

```
ip address show enp0s1
```

- `state UP` 인지
- `LOWER_UP` 이 있는지
- `inet 192.168.x.x/24` 같은 IPv4 주소가 붙었는지
- 필요하면 IPv6 주소도 붙었는지

`ip address show enp1s0` 출력으로 인터페이스의 상태(`UP`, `LOWER_UP`)와 `inet`, `inet6` 주소를 함께 확인한다. 즉, 이 명령은 "선이 꽂혀 있고, 인터페이스가 살아 있으며, 주소도 받았는가"를 한 번에 보는 용도

- `UP` 이 아니면 인터페이스 자체가 비활성일 수 있음
- `LOWER_UP` 이 없으면 케이블/가상 NIC 링크 문제일 수 있음
- `inet` 이 없으면 IPv4 주소를 못 받은 상태일 수 있음

1-2. `ip route`

패킷이 어디로 나갈지 보는 도구

```
ip route show default
```

문서에서는 기본 게이트웨이 확인용으로 이 명령을 제시하고, 예시 출력도 `default via 192.0.2.254 dev enp1s0 ...` 형태로 보여준다. IPv6 기본 경로는 `ip -6 route show default` 로 본다.

- `default via 192.168.0.1 dev enp0s1`
  - "내가 모르는 목적지는 일단 192.168.0.1로 보내라"
- 이 줄이 없으면
  - 같은 대역 로컬 통신은 될 수도 있지만
  - 외부 인터넷은 보통 안 됨

1-3. 정책 라우팅까지 갈 때의 `ip`

Red Hat 문서에는 더 고급 예시로 `ip rule list`, `ip route list table 5000` 같은 것도 나온다. 이건 단순 기본 경로 말고 **특정 서브넷은 다른 게이트웨이로 보내는 식의 고급 라우팅**을 볼 때 쓰는 도구다. 지금 단계에서는 우선 `ip address`, `ip route show default` 가 가장 중요하다.

2. `ping` : 실제로 닿는지 확인하는 도구

Red Hat 문서에서는 연결 검증에 `ping` 을 사용해 **이 호스트가 다른 호스트에 패킷을 보낼 수 있는지 확인**하라고 안내한다. 또 `connection.ip-ping-addresses` 설정을 통해 특정 IP에 실제 응답이 오는지 확인해 네트워크가 완전히 올라왔는지도 검사할 수 있다고 설명한다.

```
ping-c3192.168.0.1
ping-c38.8.8.8
ping-c3 google.com
```

2-1. 게이트웨이로 `ping`

```
ping-c3192.168.0.1
```

이건 **같은 로컬 네트워크 안에서 라우터(게이트웨이)까지 닿는지 확인**

성공하면:

- 인터페이스는 대체로 정상
- IP/서브넷 설정도 크게 틀리지 않았을 가능성이 큼
- 최소한 로컬 네트워크 연결은 살아 있음

실패하면:

- IP 대역이 틀렸거나
- 케이블/가상 NIC 문제거나
- 게이트웨이 주소를 잘못 넣었을 수 있음

2-2. 외부 IP로 `ping`

```
ping-c38.8.8.8
```

DNS를 거치지 않고 인터넷까지 나가는지 확인

성공하면:

- 게이트웨이/라우팅은 대체로 정상
- 외부 네트워크로 나가는 경로가 있음

실패하면:

- 게이트웨이는 있어도 상위 네트워크 문제
- NAT/방화벽/라우팅 문제 가능

2-3. 도메인으로 `ping`

```
ping-c3 google.com
```

DNS 이름 해석까지 포함해서 테스트

- `8.8.8.8` 은 되는데 `google.com` 이 안 되면 거의 DNS 문제
- 둘 다 안 되면 라우팅이나 외부 연결 자체가 문제일 가능성이 큼

3. `ss` : 어떤 서비스가 어떤 포트에서 듣고 있는지 확인

열린 소켓, 리스닝 포트, 연결 상태를 보는 도구

Red Hat 문서에서는 DNS 예시에서 `dnsmasq` 가 실제로 `127.0.0.1:53` 에서 리스닝 중인지 `ss -tulpn | grep "127.0.0.1:53"` 으로 확인하게 한다. 즉, `ss` 는 "설정만 한 게 아니라 프로세스가 실제로 포트를 열고 있는가"를 검증하는 도구로 쓰인다.

```
ss-tulpn
```

각 옵션 의미

- `t` : TCP
- `u` : UDP
- `l` : LISTEN 중인 소켓
- `p` : 어떤 프로세스인지 표시
- `n` : 숫자로 표시
- 예시

  ```
  tcp LISTEN01280.0.0.0:220.0.0.0:* users:(("sshd",pid=...,fd=...))
  ```

  - SSH 서버가 떠 있음
  - 22번 포트에서 접속을 기다리는 중
  - 외부에서 들어오는 연결을 받을 준비가 됨

  반대로 서버가 안 열린 것 같을 때 `ss` 에 아무것도 안 보이면

  - 서비스가 안 떠 있거나
  - 다른 포트로 떠 있거나
  - 로컬 바인딩만 했을 가능성 O

즉 `ss` 는 **내 시스템 안의 네트워크 서비스가 제대로 열려 있나"** 를 보는 도구

4. DNS 상태 점검: `/etc/resolv.conf` , `ss` , `journalctl` , `tcpdump`

DNS 흐름

```
google.com 입력
   ↓
내 컴퓨터가 DNS 서버에 물어봄
   ↓
DNS 서버가 IP 알려줌
   ↓
그 IP로 통신 시작
```

그래서 문제는 3가지 중 하나

1. DNS 서버 설정이 틀림
2. DNS 요청이 안 나감
3. DNS 응답이 안 돌아옴

→ 이걸 확인하는 도구들이 `/etc/resolv.conf`, `ss`, `journalctl`, `tcpdump`

4-1. `/etc/resolv.conf`

- 내가 사용할 DNS 서버 목록

```
cat /etc/resolv.conf
```

Red Hat 문서의 이더넷 연결 검증 절차에서도 DNS 설정 확인용으로 이 파일을 보라고 한다. 여기에 `nameserver` 가 어떤 값인지 보면 현재 시스템이 어느 DNS 서버를 쓰는지 알 수 있다. 여러 활성 프로필이 있으면 순서는 DNS 우선순위와 연결 유형의 영향을 받을 수 있다고 문서가 설명한다.

- 만약 `ping google.com` 이 안돼서 `cat /etc/resolv.conf` 해보니까 결과가 안나왔다는건 DNS 설정 자체가 안되었다는 뜻, 이걸 확인하는 용도

4-2. `journalctl`

- 왜 안되는지 보기위해서 내부 로그를 보는 명령어

```
journalctl -xeu NetworkManager -n 50
```

- 옵션 의미
  - `x` : 에러/경고에 대한 추가 설명
  - `e` : 로그 끝부분으로 이동
  - `u NetworkManager` : NetworkManager 서비스만 보기
  - `n 50` : 최근 50줄만 보기

NetworkManager가 네트워크를 어떻게 처리했는지 로그를 보는 명령

Red Hat 문서도 DNS 문제 해결 시 `journalctl -u NetworkManager` 나 `journalctl -xeu NetworkManager` 로 어떤 DNS 서버가 어떤 도메인에 사용되는지 확인하라고 안내한다.

즉 설정은 맞아 보이는데 동작이 이상할 때는 **"실제 런타임에서 NetworkManager가 무슨 결정을 했는가"** 를 로그로 보는 것 !

4-3. `tcpdump`

- 실제 패킷이 오가는걸 보는 도구

```
tcpdump-i any port53
```

- 옵션 의미
  - `i any`
    - 모든 인터페이스에서 패킷 보기
    - `enp0s1` 만이 아니라 전체 인터페이스 대상
  - `port 53`
    - 53번 포트만 필터링
    - 즉 DNS 트래픽만 보겠다는 뜻

Red Hat 문서에서는 DNS 요청이 정말 의도한 인터페이스와 DNS 서버로 나가는지 `tcpdump` 로 검증하는 예를 보여준다.

- `google.com` 조회를 했을 때
- 실제 패킷이 어느 인터페이스로 나가는지
- 어느 DNS 서버로 보내는지

실습

```
1) 인터페이스 상태 확인 및 IP 주소 확인
2) 기본 게이트웨이 확인
3) DNS 확인
4) 실제 통신 확인(ping)
5) 포트/서비스 확인(ss)
6) 로그 확인(journalctl)
7) 패킷 확인(tcpdump)
```

1. 인터페이스 상태 확인 및 IP 주소 확인

```
ip address show enp0s1
```

```
dldbstj@RHEL8:~$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 22:5a:f8:47:24:24 brd ff:ff:ff:ff:ff:ff
    altname enx225af8472424
dldbstj@RHEL8:~$ ip address show enp0s1
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 22:5a:f8:47:24:24 brd ff:ff:ff:ff:ff:ff
    altname enx225af8472424
    inet 192.168.64.50/24 brd 192.168.64.255 scope global noprefixroute enp0s1
       valid_lft forever preferred_lft forever
    inet 192.168.64.3/24 brd 192.168.64.255 scope global secondary dynamic noprefixroute enp0s1
       valid_lft 2406sec preferred_lft 2406sec
    inet6 fd7e:7ced:7640:9bde:205a:f8ff:fe47:2424/64 scope global dynamic mngtmpaddr proto kernel_ra
       valid_lft 2591916sec preferred_lft 604716sec
    inet6 fe80::205a:f8ff:fe47:2424/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

`UP` : 인터페이스가 켜져 있다는 뜻

`LOWER_UP` : 실제로 링크가 살아 있다는 뜻 , 가상머신이면 “가상 NIC가 연결됨”, 물리 서버면 “케이블 연결됨”에 가까움

`inet` : IPv4 주소가 붙음

`inet6` : IPv6 주소가 붙음

```
ip -br address
```

```
dldbstj@RHEL8:~$ ip -br address
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s1           UP             192.168.64.50/24 192.168.64.3/24 fd7e:7ced:7640:9bde:205a:f8ff:fe47:2424/64 fe80::205a
:f8ff:fe47:2424/64
```

2. 게이트 웨이 확인

```
ip route show default
```

```
dldbstj@RHEL8:~$ ip route show default
default via 192.168.64.1 dev enp0s1 proto static metric 100
```

- `default`
  → 목적지를 따로 모르면 기본으로 이 경로 사용
- `via 192.168.64.1`
  → 게이트웨이 주소는 `192.168.64.1`
- `dev enp0s1`
  → `enp0s1` 인터페이스로 보냄

3. DNS 설정 확인

```
cat /etc/resolv.conf
```

```
dldbstj@RHEL8:~$ cat /etc/resolv.conf
# Generated by NetworkManager
nameserver 8.8.8.8
nameserver 1.1.1.1
nameserver 192.168.64.1
```

- `search`
  → 도메인 검색 suffix
- `nameserver`
  → DNS 서버 주소

4. 실제 연결 확인 - ping 3단계

1단계 : 게이트웨이까지 가는지 확인

```
ping -c 3 192.168.64.1
```

```
dldbstj@RHEL8:~$ ping -c 3 192.168.64.1
PING 192.168.64.1 (192.168.64.1) 자료의 56(84) 바이트.
64 바이트 (192.168.64.1에서): icmp_seq=1 ttl=64 시간=1.80 ms
64 바이트 (192.168.64.1에서): icmp_seq=2 ttl=64 시간=1.64 ms
64 바이트 (192.168.64.1에서): icmp_seq=3 ttl=64 시간=0.981 ms

--- 192.168.64.1 ping 통계 ---
3 패킷이 전송됨, 3 수신됨, 0% packet loss, time 2009ms
rtt min/avg/max/mdev = 0.981/1.473/1.803/0.354 ms
```

- 링크 정상
- IP/서브넷 정상일 가능성 큼
- 로컬 네트워크 연결 정상

2단계 : 외부 IP까지 가는거 확인

```
ping -c 3 8.8.8.8
```

```
dldbstj@RHEL8:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 자료의 56(84) 바이트.
64 바이트 (8.8.8.8에서): icmp_seq=1 ttl=114 시간=36.3 ms
64 바이트 (8.8.8.8에서): icmp_seq=2 ttl=114 시간=33.9 ms
64 바이트 (8.8.8.8에서): icmp_seq=3 ttl=114 시간=35.7 ms

--- 8.8.8.8 ping 통계 ---
3 패킷이 전송됨, 3 수신됨, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 33.883/35.293/36.305/1.028 ms
```

- 외부 인터넷 경로 있음
- 게이트웨이/라우팅 대체로 정상

3단계 : 도메인 이름까지 되는지 확인

```
ping -c 3 google.com
```

```
dldbstj@RHEL8:~$ ping -c 3 google.com
PING google.com (172.217.209.102) 자료의 56(84) 바이트.
64 바이트 (hq-in-f102.1e100.net (172.217.209.102)에서): icmp_seq=1 ttl=111 시간=34.6 ms
64 바이트 (hq-in-f102.1e100.net (172.217.209.102)에서): icmp_seq=2 ttl=111 시간=74.3 ms
64 바이트 (hq-in-f102.1e100.net (172.217.209.102)에서): icmp_seq=3 ttl=111 시간=120 ms

--- google.com ping 통계 ---
3 패킷이 전송됨, 3 수신됨, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 34.619/76.144/119.533/34.691 ms
```

5. ss로 포트와 서비스 확인

```
ss - tulpn
```

```
dldbstj@RHEL8:~$ ss -tulpn
Netid       State       Recv-Q      Send-Q          Local Address:Port           Peer Address:Port      Process
udp         UNCONN      0           0                  127.0.0.1:323                0.0.0.0:*
udp         UNCONN      0           0                    0.0.0.0:5353               0.0.0.0:*
udp         UNCONN      0           0                      [::1]:323                   [::]:*
udp         UNCONN      0           0                      [::]:5353                   [::]:*
tcp         LISTEN      0           4096               127.0.0.1:631                0.0.0.0:*
tcp         LISTEN      0           128                  0.0.0.0:22                 0.0.0.0:*
tcp         LISTEN      0           4096                   [::1]:631                   [::]:*
tcp         LISTEN      0           4096                       *:9090                     *:*
tcp         LISTEN      0           128                     [::]:22                    [::]:*
```

- `t` : TCP
- `u` : UDP
- `l` : LISTEN/대기 중
- `p` : 프로세스 표시
- `n` : 숫자로 표시

현재 DNS 리스닝 확인 실습

```
dldbstj@RHEL8:~$ sudo dnf install dnsmasq
[sudo] dldbstj 암호:
서브스크립션 관리 저장소를 최신화하기.
마지막 메타자료 만료확인(1:24:32 이전): 2026년 04월 06일 (월) 오후 06시 07분 30초.
꾸러미 dnsmasq-2.90-4.el10.aarch64가 이미 설치되어 있습니다.
종속성이 해결되었습니다.
처리가 필요하지 않습니다.
완료되었습니다!
dldbstj@RHEL8:~$ sudo systemctl start dnsmasq
dldbstj@RHEL8:~$
dldbstj@RHEL8:~$
dldbstj@RHEL8:~$ ss -tulpn | grep :53
udp   UNCONN 0      0            0.0.0.0:5353      0.0.0.0:*
udp   UNCONN 0      0          127.0.0.1:53        0.0.0.0:*
udp   UNCONN 0      0              [::]:5353         [::]:*
udp   UNCONN 0      0             [::1]:53           [::]:*
tcp   LISTEN 0      32         127.0.0.1:53        0.0.0.0:*
tcp   LISTEN 0      32            [::1]:53           [::]:*
```

- 설치

  ```
  sudo dnf install dnsmasq
  ```

- 실행

  ```
  sudo systemctl start dnsmasq
  ```

- 확인

  ```
  ss -tulpn | grep :53
  ```

  53번 포트가 LISTEN이면 = DNS 서버가 돌아가는 상태

6. journalctl로 networkmanager 로그 확인
   - 현재 연결 프로필 이름 확인

     ```
     nmcli connection show
     ```

     ```
     dldbstj@RHEL8:~$ nmcli connection show
     NAME    UUID                                  TYPE      DEVICE
     enp0s1  80845e61-5a51-3cba-a0e1-3ed8c641e6ad  ethernet  enp0s1
     lo      30250a86-1d6c-4b51-9da1-983b34417cf4  loopback  lo
     dldbstj@RHEL8:~$
     dldbstj@RHEL8:~$
     dldbstj@RHEL8:~$ journalctl -u NetworkManager
     ```

   - 기존 로그 보기

     ```
     journalctl -u NetworkManager
     ```

     - 지금까지 NetworkManager가 남긴 로그 확인
     - 연결 성공/실패 흔적 확인 가능

```
dldbstj@RHEL8:~$ journalctl -u NetworkManager
Failed to find catalog entry: 부적절한 인수
Failed to find catalog entry: 부적절한 인수
Failed to find catalog entry: 부적절한 인수
 3월 22 21:27:09 RHEL8 systemd[1]: Starting NetworkManager.service - Network Manager...
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9307] NetworkManager (version 1.52.0-1.el10_0) is sta
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9310] Read config: /etc/NetworkManager/NetworkManager
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9368] manager[0xaaaac8627d60]: monitoring kernel firm
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9777] hostname: hostname: using hostnamed
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9778] hostname: static hostname changed from (none) t
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9783] dns-mgr: init: dns=default,systemd-resolved rc-
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9848] manager[0xaaaac8627d60]: rfkill: Wi-Fi hardware
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9848] manager[0xaaaac8627d60]: rfkill: WWAN hardware
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9896] Loaded device plugin: NMBluezManager (/usr/lib6
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9915] Loaded device plugin: NMAtmManager (/usr/lib64/
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9920] Loaded device plugin: NMWwanFactory (/usr/lib64
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9934] Loaded device plugin: NMWifiFactory (/usr/lib64
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9944] manager: rfkill: Wi-Fi enabled by radio killswi
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9945] manager: rfkill: WWAN enabled by radio killswit
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9945] manager: Networking is enabled by state file
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9954] settings: Loaded settings plugin: keyfile (inte
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9990] dhcp: init: Using DHCP client 'internal'
 3월 22 21:27:09 RHEL8 NetworkManager[887]: <info>  [1774182429.9992] manager: (lo): new Loopback device (/org/freede
 3월 22 21:27:10 RHEL8 NetworkManager[887]: <info>  [1774182430.0007] device (lo): state change: unmanaged -> unavail
 3월 22 21:27:10 RHEL8 NetworkManager[887]: <info>  [1774182430.0009] device (lo): state change: unavailable -> disco
 3월 22 21:27:10 RHEL8 NetworkManager[887]: <info>  [1774182430.0012] device (lo): Activation: starting connection 'l
 3월 22 21:27:10 RHEL8 NetworkManager[887]: <info>  [1774182430.0023] manager: (enp0s1): new Ethernet device (/org/fr
 3월 22 21:27:10 RHEL8 NetworkManager[887]: <info>  [1774182430.0025] device (enp0s1): state change: unmanaged -> unav
```

- 새 로그가 생기도록 연결 다시 올리기

```
sudo nmcli connection up "Wired connection 1"
journalctl -xeu NetworkManager -n 50
```

```
dldbstj@RHEL8:~$ sudo nmcli connection up "Wired connection 1"
journalctl -xeu NetworkManager -n 50
[sudo] dldbstj 암호 :
오류: 알 수 없는 연결 'Wired connection 1'.
 4월 06 18:07:54 RHEL8 NetworkManager[887]: <info>  [1775466474.3167] device (enp0s1): state change: ip-config -> ip-
 4월 06 18:07:54 RHEL8 NetworkManager[887]: <info>  [1775466474.3193] device (enp0s1): state change: ip-check -> seco
 4월 06 18:07:54 RHEL8 NetworkManager[887]: <info>  [1775466474.3197] device (enp0s1): state change: secondaries -> a
 4월 06 18:07:54 RHEL8 NetworkManager[887]: <info>  [1775466474.3203] manager: NetworkManager state is now CONNECTED_
 4월 06 18:07:54 RHEL8 NetworkManager[887]: <info>  [1775466474.3210] device (enp0s1): Activation: successful, device
 4월 06 18:07:54 RHEL8 NetworkManager[887]: <info>  [1775466474.3223] manager: NetworkManager state is now CONNECTED_
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.2681] audit: op="connection-update" uuid="80045e61-5a
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.3245] agent-manager: agent[bcb36c9f8d7ed090,:1.850/ne
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.3251] device (enp0s1): state change: activated -> dea
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.3256] manager: NetworkManager state is now DISCONNECT
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.3276] device (enp0s1): disconnecting for new activati
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.3279] audit: op="connection-activate" uuid="80045e61-
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.3697] device (enp0s1): state change: deactivating -> 
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.4253] manager: NetworkManager state is now DISCONNECT
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.4265] device (enp0s1): Activation: starting connectio
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.4294] device (enp0s1): state change: disconnected -> 
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.4296] manager: NetworkManager state is now CONNECTING
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.4310] device (enp0s1): state change: prepare -> confi
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.4478] device (enp0s1): state change: config -> ip-con
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.4483] dhcp4 (enp0s1): activation: beginning transacti
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.4514] policy: set 'enp0s1' (enp0s1) as default for IP
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.5083] dhcp4 (enp0s1): state changed new lease, addres
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.6696] dhcp4 (enp0s1): state changed new lease, addres
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.6791] device (enp0s1): state change: ip-config -> ip-
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.6839] device (enp0s1): state change: ip-check -> seco
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.6840] device (enp0s1): state change: secondaries -> a
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.6852] manager: NetworkManager state is now CONNECTED_
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.6860] device (enp0s1): Activation: successful, device
 4월 06 18:14:10 RHEL8 NetworkManager[887]: <info>  [1775466850.6863] manager: NetworkManager state is now CONNECTED_
```

새 터미널에서 생성시키고 방금 생성된 로그 최신 50줄 보기

7. tcpdump로 DNS 패킷 보기

- 설치

```
sudo dnf install tcpdump
```

```
dldbstj@RHEL8:~$ sudo dnf install tcpdump
서브스크립션 관리 저장소를 최신화하기.
마지막 메타자료 만료확인(1:27:11 이전): 2026년 04월 06일 (월) 오후 06시 07분 30초.
꾸러미 tcpdump-14:4.99.4-10.el10.aarch64가 이미 설치되어 있습니다.
종속성이 해결되었습니다.
처리가 필요하지 않습니다.
완료되었습니다!
```

- 현재 DNS 설정 미리 확인

```
cat /etc/resolv.conf
```

```
dldbstj@RHEL8:~$ cat /etc/resolv.conf
# Generated by NetworkManager
nameserver 8.8.8.8
nameserver 1.1.1.1
nameserver 192.168.64.1
```

- **첫 번째 터미널에서 DNS 패킷 캡처 시작**

```
sudo tcpdump -i any port 53
```

```
dldbstj@RHEL8:~$ sudo tcpdump -i any port 53
tcpdump: data link type LINUX_SLL2
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
```

  - 모든 인터페이스에서 DNS 트래픽 감시 시작
  - DNS 요청/응답이 실시간으로 보임
- 다른 터미널에서 DNS 요청 발생시키기

```
ping -c 1 google.com
```

```
dldbstj@RHEL8:~$ ping -c 1 google.com
PING google.com (172.217.209.139) 자료의 56(84) 바이트.
64 바이트 (hq-in-f139.1e100.net (172.217.209.139)에서): icmp_seq=1 ttl=111 시간=31.9 ms

--- google.com ping 통계 ---
1 패킷이 전송됨, 1 수신됨, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 31.902/31.902/31.902/0.000 ms
```

```
dldbstj@RHEL8:~$ sudo tcpdump -i any port 53
[sudo] dldbstj 암호:
tcpdump: data link type LINUX_SLL2
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
19:38:13.654382 enp0s1 Out IP RHEL8.33817 > dns.google.domain: 45572+ A? google.com. (28)
19:38:13.654420 enp0s1 Out IP RHEL8.33817 > dns.google.domain: 29696+ AAAA? google.com. (28)
19:38:13.674839 enp0s1 Out IP RHEL8.43184 > dns.google.domain: 36305+ PTR? 50.64.168.192.in-addr.arpa. (44)
19:38:13.691413 enp0s1 In  IP dns.google.domain > RHEL8.33817: 45572 6/0/0 A 172.217.209.100, A 172.217.209.139
, A 172.217.209.138, A 172.217.209.113, A 172.217.209.102, A 172.217.209.101 (124)
19:38:13.709670 enp0s1 In  IP dns.google.domain > RHEL8.43184: 36305 NXDomain 0/0/0 (44)
19:38:13.710947 enp0s1 Out IP RHEL8.35367 > dns.google.domain: 38218+ PTR? 8.8.8.8.in-addr.arpa. (38)
19:38:13.734642 enp0s1 In  IP dns.google.domain > RHEL8.33817: 29696 4/0/0 AAAA 2404:6800:400a:1000::71, AAAA 2
404:6800:400a:1000::66, AAAA 2404:6800:400a:1000::8a, AAAA 2404:6800:400a:1000::64 (140)
19:38:13.743623 enp0s1 In  IP dns.google.domain > RHEL8.35367: 38218 1/0/0 PTR dns.google. (62)
19:38:13.768617 enp0s1 Out IP RHEL8.56403 > dns.google.domain: 63085+ PTR? 100.209.217.172.in-addr.arpa. (46)
19:38:13.804093 enp0s1 In  IP dns.google.domain > RHEL8.56403: 63085 1/0/0 PTR hq-in-f100.1e100.net. (80)
```

방화벽 정책 설정

`firewalld` : "무엇을 허용하고 무엇을 막을지 정하는 작업"

- `firewalld` 는 규칙을 단순히 포트 번호만으로 관리하지 않고 **zone / service / port / rich rule / policy object** 같은 개념으로 관리한다
- **인터페이스를 zone에 넣고 그 zone에 서비스/포트/세부 규칙을 붙여서 허용/차단 정책을 만드는 도구**

방화벽 정책 설정이 필요한 이유

- 서버에 열려 있는 모든 포트를 아무나 접근 가능하게 두면 위험함
- 꼭 필요한 서비스만 열어야 함
- 네트워크 위치에 따라 다른 보안 수준을 적용해야 함
  - 예: 외부망은 엄격하게
  - 내부망은 조금 더 유연하게
- 운영 중인 서비스에 맞춰**SSH만 허용**, **웹만 허용**, **특정 IP만 허용** 같은 정책이 필요함

zone

- **네트워크의 신뢰 수준별 규칙 묶음**
- 인터페이스나 소스 IP를 특정 zone에 넣고 그 zone에 허용 정책을 붙이는 구조
- 왜 필요할까 ?
  - 같은 서버라도 연결된 네트워크가 다를 수 있음
  - 외부 인터넷에 연결된 NIC와 내부 사설망 NIC는 같은 보안 정책이면 안 됨
- 예시
  - `public`
    - 외부망
    - 필요한 것만 최소 허용
  - `internal`
    - 내부망
    - 상대적으로 더 신뢰
  - `dmz`
    - 외부 공개 서버용

service

- 자주 쓰는 네트워크 서비스를**이름으로 묶어둔 사전 정의 규칙**
- 예:
  - `ssh` 허용 : 내부적으로 SSH에 필요한 포트/프로토콜을 허용
  - `http` 허용 : 내부적으로 웹 서비스에 필요한 포트 허용
  - `https` 허용 : 내부적으로 웹 서비스에 필요한 포트 허용

port

- 서비스를 이름으로 열지 않고 **직접 포트 번호로 허용** 하는 방식
- 커스텀 애플리케이션
- 사전 정의 서비스가 없는 경우
- 예:
  - `8080/tcp`
  - `9000/tcp`

    - `5353/udp`
- service와의 차이
    - service
        - 의미 단위
        - 사람이 읽기 좋음
    - port
        - 숫자 단위
        - 직접적이지만 관리가 덜 편함

rich rule

- 단순히 "포트를 열기"보다 더 복잡한 조건을 줄 수 있는 규칙
- 예:
    - 특정 IP 대역만 허용
    - 나머지는 drop
    - 특정 서비스에 대해 특정 소스만 허용
- `rpc.mountd` 보안 예시

runtime

- 지금 즉시 적용되는 현재 설정
- 테스트하기 좋음
- 하지만 reload/restart 후 유지되지 않을 수 있음

permanent

- 영구 저장되는 설정
- 재부팅 후에도 유지됨
- 실습할 때는 일단 runtime으로 테스트
- 문제 없으면 permanent로 저장

policy object

- zone 하나 내부의 규칙이 아니라 **zone과 zone 사이를 지나는 트래픽 정책** 을 다루는 객체
- 언제 중요할까 ?
    - 라우팅
    - 포워딩
    - 여러 네트워크 세그먼트가 있는 환경
    - 컨테이너/브리지/멀티 NIC 서버

서비스 허용과 포트 허용의 차이

- 서비스 허용
    - `ssh`, `http` 처럼 의미 단위
    - 더 읽기 쉽고 관리 쉬움
- 포트 허용
    - `8080/tcp` 처럼 직접 숫자 지정
    - 커스텀 앱에 적합

실습

- 현재 어떤 zone이 적용되어 있는지 확인
- 서비스 단위로 허용하기
- 포트 단위로 허용하기
- 특정 조건만 허용하는 rich rule 적용

1. firewalld 상태 확인
    - 방화벽 서비스가 실제 실행 중인지 확인

    ```
    sudo systemctl status firewalld
    sudo firewall-cmd --state
    ```

2. 현재 방화벽 전체 상태 확인

    ```
    sudo firewall-cmd --get-active-zones
    sudo firewall-cmd --get-default-zone
    ```

```
sudo firewall-cmd --list-all
```

`-get-active-zones:` 현재 활성화된 zone과 그 zone에 연결된 인터페이스를 보여줌

`-get-default-zone :` 기본 zone이 무엇인지 보여줌

`-list-all:` 현재 zone에 어떤 서비스/포트/설정이 들어 있는지 보여줌

3. SSH 서비스 허용

```
sudo firewall-cmd --add-service=ssh
sudo firewall-cmd --list-all
```

- 기본 zone에 ssh 서비스 런타임으로 추가
- 즉시 적용되지만 영구 저장 아직 아님
- 영구 저장

```
sudo firewall-cmd --runtime-to-permanent
sudo firewall-cmd --list-all --permanent
```

4. HHTP 서비스 허용

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

5. 커스텀 포트 8080 열기

```
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

6. 특정 zone에서만 규칙 주기

```
sudo firewall-cmd --zone=public --permanent --add-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --zone=public --list-all
```

7. 인터페이스를 zone에 연결하기

```
ip a
sudo firewall-cmd --get-active-zones
```

```
sudo firewall-cmd --zone=internal --change-interface=enp0s1 --permanent
sudo firewall-cmd --reload
sudo firewall-cmd --get-active-zones
```

8. rich rule로 특정 대역만 허용

```
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="192.168.0.0/24" service name="ssh" a
ccept'
sudo firewall-cmd --reload
sudo firewall-cmd --list-rich-rules
```

9. 규칙 제거

```
sudo firewall-cmd --remove-service=ssh
sudo firewall-cmd --permanent --remove-port=8080/tcp
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

SELinux 동작 방식과 기본 보안 정책 이해

SELinux란

- SELinux(Security-Enhanced Linux)는 리눅스 커널에 포함된 보안 기능
- 기존 리눅스 권한(DAC: rwx)에 더해 추가적인 보안 정책(MAC)을 적용
- 프로그램이 “허용된 범위 내에서만” 동작하도록 강제
- 왜 필요한가
  - 기존 리눅스는 권한만 맞으면 접근 가능
  - SELinux는 권한이 맞더라도 정책이 허용하지 않으면 차단
  - 서비스가 해킹되더라도 시스템 전체로 피해 확산을 막는 역할

동작 방식

- 프로세스가 파일, 포트 등에 접근 시도
- 커널이 SELinux 정책 확인
- 허용된 경우 → 접근 가능
- 허용되지 않은 경우 → 차단 + 로그 기록

핵심 특징

- 기본은 거부(Default Deny)
- 허용된 동작만 가능

1. Label

- 모든 파일과 프로세스에는 label이 붙어 있음
- 형식: user:role:type:level
- 실제로 중요한 것은 type

예시

- 프로세스: httpd_t (웹 서버)
- 파일: httpd_sys_content_t (웹 콘텐츠)

의미

- httpd 프로세스는 httpd용 파일만 접근 가능

---

2. Domain과 Type

- 프로세스의 type → domain
- 파일의 type → file type

접근 제어 기준

- 어떤 domain이 어떤 type에 접근 가능한가

기본 보안 정책 - Targeted 정책

- RHEL 기본 정책
- 주요 서비스만 보호

보호 대상 예시

- 웹 서버 (httpd)
- SSH
- 데이터베이스

일반 사용자 프로그램은 제한이 거의 없음

정책 원칙

최소 권한 원칙 : 필요한 권한만 허용

격리 : 서비스 간 접근 제한

SELinux 모드

Enforcing

- 정책을 실제로 적용
- 허용되지 않은 접근은 차단

Permissive

- 차단하지 않고 로그만 기록
- 디버깅 용도

Disabled

- SELinux 완전 비활성화
- 사용 권장하지 않음

자주 발생하는 문제 상황

파일 권한은 맞는데 접근이 안 되는 경우

- SELinux 정책에 의해 차단된 것

웹 서버 디렉터리 변경

- 파일 label이 맞지 않아 접근 실패

포트 변경

- SELinux가 해당 포트를 허용하지 않음

chmod 777 해도 안 되는 경우

- SELinux 문제

Boolean

- SELinux 정책의 on/off 스위치
- 특정 기능 허용 여부를 설정

예시

- 웹 서버가 DB 접근 허용 여부

명령어

```
setsebool -P httpd_can_network_connect_db on
```

문제 해결 흐름

1. SELinux 상태 확인

   ```
   getenforce
   ```

2. 테스트용 permissive 전환

   ```
   setenforce 0
   ```

3. 정상 동작 시 SELinux 문제로 판단
4. 원인 분석
   - 파일 label 문제
   - 포트 문제
   - Boolean 설정 문제
5. 해결 후 다시 enforcing 복귀

   ```
   setenforce 1
   ```

주요 명령어

상태 확인

```
getenforce
sestatus
```

컨텍스트 확인

```
ls -Z
id -Z
```

파일 label 복구

```
restorecon -Rv /경로
```

Boolean 확인

```
getsebool -a
```

포트 확인

```
semanage port -l
```

실습

1. SELinux 상태 확인

```
getenforce
sestatus
```

- 현재 enforcing 상태 확인

---

2. permissive 모드로 전환 테스트

```
setenforce 0
```

- 서비스 동작 확인
- 정상 동작 시 SELinux가 원인

---

3. 다시 enforcing 모드 복귀

```
setenforce 1
```

---

4. 파일 label 확인

```
ls -Z /var/www/html
```

- 웹 서버 파일의 label 확인

5. label 복구

```
restorecon -Rv /var/www/html
```

- 잘못된 label을 기본값으로 복구

6. Boolean 설정 확인 및 변경

```
getsebool -a
setsebool -P httpd_can_network_connect_db on
```

7. 포트 허용 확인

```
semanage port -l
```

요약

- 지금 네트워크가 제대로 설정됐는지 "확인"하는 것
  - 인터페이스 살아있나? `ip address` 에서 `UP`, `LOWER_UP`
  - IP 붙었나? `ip address`에서 `inet 192.168.x.x`
  - 게이트웨이 있나? `ip route` 에서 `default via 192.168.x.x`
  - 어떤 서비스가 열려있나 ? `ss`
  - DNS 있나? `cat /etc/resolv.conf` 에서 `nameserver 8.8.8.8`
  - 문제가 있는게 어디일까 ? 로그 보는 건 `journalctl` , 패킷 보는건 `tcpdump`
  - 진짜 통신 되나? `ping`
- 누가 들어올 수 있는지 "통제"하는 것 : `firewalld` / `firewall-cmd`
  - SSH 열까?
  - HTTP 열까?
  - 8080 포트 열까?
  - 특정 IP만 허용할까

```
zone        → 어디서 들어오냐
service     → 어떤 서비스냐
port        → 어떤 문이냐
rich rule   → 누가 들어오냐
runtime     → 지금만 적용
permanent   → 계속 적용
```